FusionOps

compiler-level operation scheduling

Team Pluto

- Anshul Badhani
- Anisha Roy

Problem Statement

Can an LLM, placed inside a simulated GPU execution environment and trained via RL, discover the same fusion, retention, and scheduling strategies that took compiler engineers years to hand-tune?

Our Hypothesis...

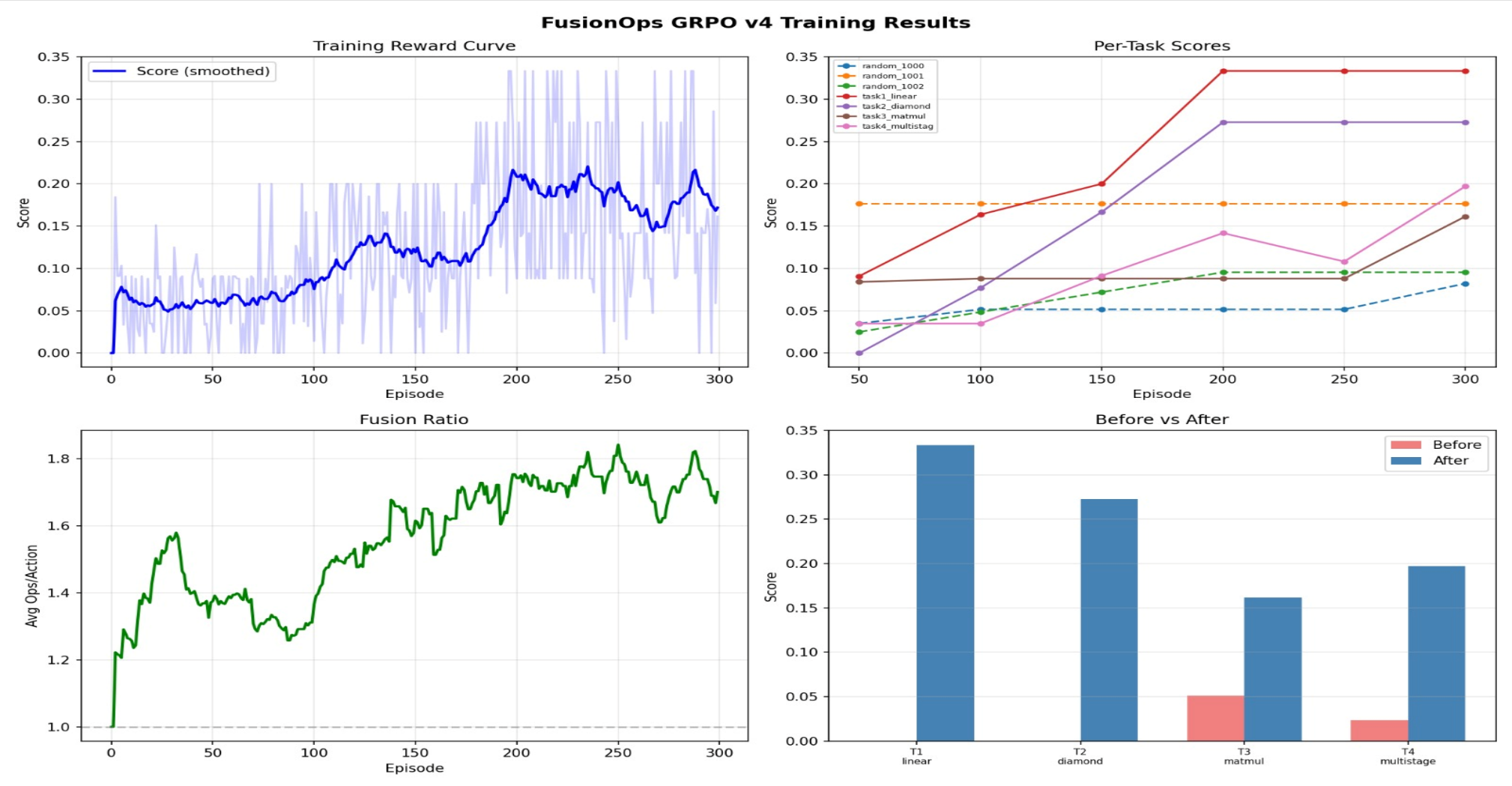
Can an LLM that is placed inside an execution environment, trained via RL discover fusion and scheduling strategies through exploration ?

v1: Without Hints

RL environment and trained on four computation graph tasks:

- Linear Chain
- Diamond Graph
- MatMul
- K-split

What we concluded after training was that, without any hints the LLM wasn't able to figure out what was the optimal way to schedule the operations.

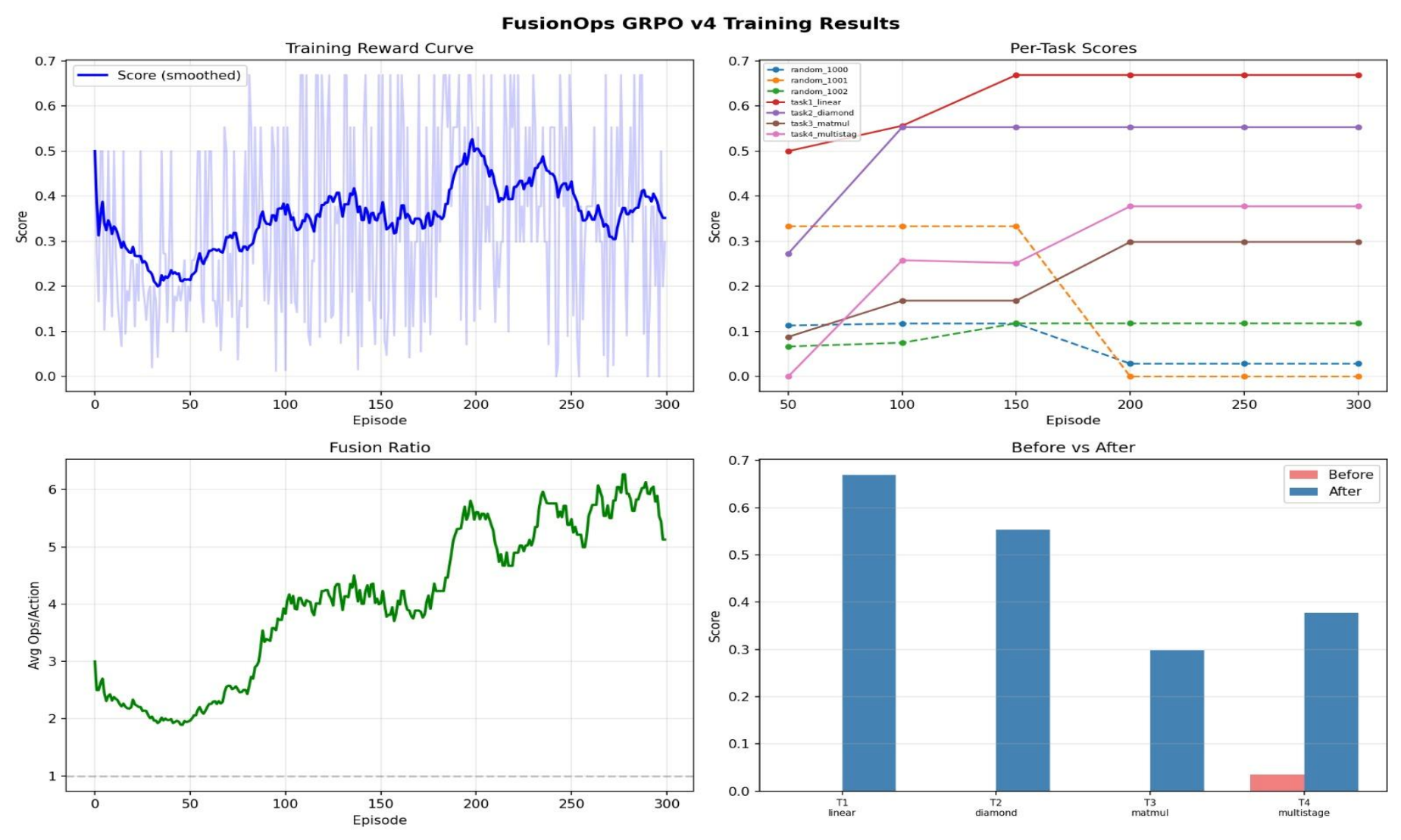


What we concluded after training was that, without any hints the LLM wasn't able to figure out what was the optimal way to schedule the operations.

v2: With Hints

Hints were introduced as a strong prior in the reward function, high weight early, progressively discounted as training advanced.

The goal was scaffolded learning: guide first, then withdraw the scaffold and force the model to stand on its own.

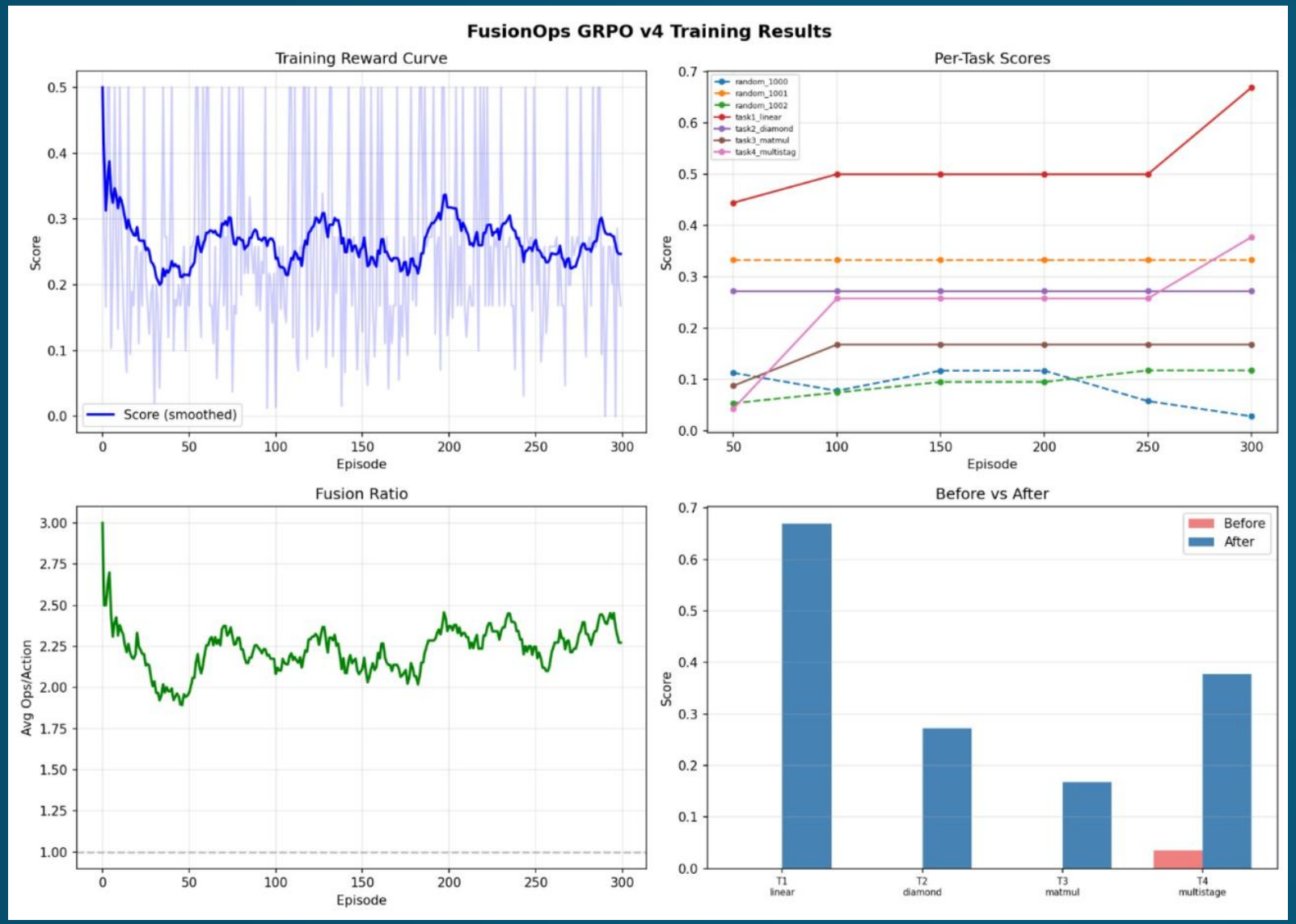


After implementation, the conclusion that we reached was, the model was able to apply the hint and fuse the operations to reduce the computation, which was our first breakthrough.

v3: With Hints and 2 Step Look Ahead Scoring

In v2, hints were rewarded immediately. In v3, we introduced a reward function that scores based on immediate reward and a discounted next step reward.

In previous version retention wasn’t achieved, so the new reward structure was introduced to unlock credit assignment for retention.



In this version despite the new structure no retention was observed.

Instead,
We gained a new insight...

The Insight

The LLM in training (Qwen2.5) was able to discover a strategy far superior to retention i.e., fusing all operations.
Hence, a sudden jump in learning towards the final episode was observed.

The hints, combined with the environment's inherent structure, *gave the model enough signal to game the evaluation rather than solve the underlying problem*. **The results looked promising, but the model had found a shortcut, not a strategy.**

This time, instead of separate tasks for computational graphs. We built a general framework where the environment is agnostic to the complexity or the shape of the graph.

Unlike before where the environment was limited to 8 operations per task, which was a hard ceiling and hence gaming was possible since, OOM error only occurs in complex computational graphs and the LLM wasn't able to learn: partial fusion, retention, and scheduling.

Thus, we built

Compiler-Scheduler.env

- Agnostic to the complexity or the shape of the graph.
- Our baseline this time is greedy as compared to the Naïve approach in FusionOps, for two reasons:
  - Greedy already performs basic fusion and immediate retention
  - In case the learning is negative. We know that, the LLM missed the easy operations that was even obvious to the greedy approach.
- Now the environment was ready to tackle the gap that the first environment failed to accomplish

Training.py

- This time instead of rewarding fusion, following the order we took a different approach

- “Do whatever reduces latency”

This means we are rewarding fusions that reduce computation time. Keeping data in fast memory (retention) and picking a good tile size.

- We used GRPO with reference anchor DPO.

In simple words

generate a few options, test them all, reward the winners, punish the losers, and copy the expert when you're still bad at it.

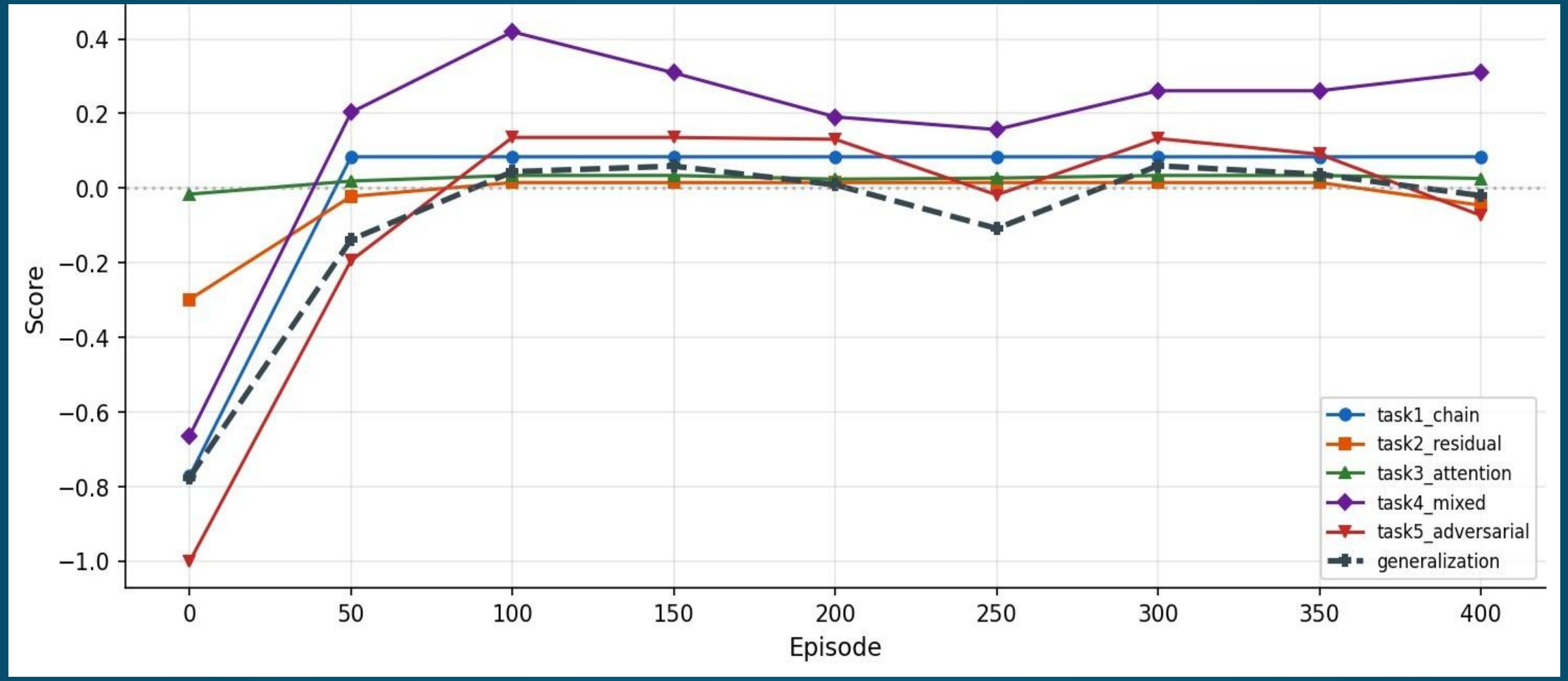
Breakthrough

We ran **400 episode** at an **interval of 50** using the new training script.

This time the LLM was actually able to generalize fusion and retention even on complex compiler operation graphs.

Which was the problem statement we started with…

Interestingly enough after 100 episodes the learning had plateaued.



Our observation is that the learning signals past the 100th episodes have become noisy.

Right now, compiler scheduling is partly solved by hardcoding done by the kernel developers and certain parameters are discovered by the user by trial and error and this amounts to around 10-30% optimization which is achieved via human intervention.

We have achieved **17% optimization in compiler latency**, purely via training an LLM in our environment in our first iteration itself.